

DEV2 – JAVL – Laboratoire Java**TD 02 – Intro OO - Partie 2***Attributs et constructeurs, types, visibilité***Résumé**

Dans ce TD vous allez ajouter à vos classes des *attributs*, et ainsi vous doter de nouveaux outils pour écrire des programmes orientés objet.

Reprendre le travail...

Pour ouvrir le projet que vous avez commencé au labo précédent, le plus sûr est

- ▷ choisir dans l'interface une option permettant d'ouvrir un projet, puis
- ▷ sélectionner le fichier `pom.xml`, et enfin
- ▷ de l'ouvrir « en tant que projet » si l'IDE vous pose la question.

Il ne faut pas...

Il faut absolument éviter d'ouvrir un autre dossier que celui qui contient le fichier `pom.xml` comme projet. Vous aurez tout un tas de problèmes bizarres que vous n'avez pas envie d'avoir.

1 Précédemment...

À la séance précédente, vous avez créé un programme orienté objet. Dans ce programme, vous avez défini le concept de chien comme "un objet qui peut aboyer".

```
/**
 * Represents a dog inside the program
 * @author tnicodeme
 */
public class Dog {
    /**
     * Tell the dog to bark
     */
    void bark() {
        System.out.println("Wouf !");
    }
}
```

Le programme en lui-même crée un chien, et le fait aboyer.

```
public class Main {
    public static void main(String[] args) {
        Dog médor = new Dog();
        médor.bark();
    }
}
```

Que se passe-t-il si on veut avoir plusieurs chiens ?

- ✍ C'est très facile ! Il suffit de créer un deuxième objet de la classe `Dog` dans notre méthode principale, et de le faire aboyer :

```
Dog milou = new Dog();
milou.bark();
```

Et lorsque vous exécutez cela :

Wouf !
Wouf !

Process finished with exit code 0

C'est embêtant ! Impossible de savoir quel chien a aboyé et à quel moment. Il serait fort pratique que lorsqu'un chien aboie, on affiche un peu plus d'informations. Quelque chose comme ceci :

Médor dit : Wouf !
Milou dit : Wouf !

Process finished with exit code 0

2 Différencier des objets de même classe

Tutoriel 1

Différencier les chiens qui aboient

Vous allez maintenant modifier la classe `Dog` afin que chaque chien possède un nom qui s'affiche lorsqu'il aboie.

Attributs

Le concept d'attribut Si les *méthodes* d'une classe définissent ce qu'un objet peut *faire*, un *attribut* désigne une information qu'un objet *connaît* à son propre sujet.

On peut voir un attribut d'un objet comme une variable définie à l'intérieur de cet objet. C'est d'ailleurs comme cela que Java les note !

- ✍ À l'intérieur de la classe `Dog` (mais en-dehors de sa méthode !), déclarez une variable qui contiendra le nom du chien :

```
String name;
```

C'est tout ! Chaque objet de la classe `Dog` va maintenant posséder un attribut **name** qui lui est propre.

Tout comme les variables en Java, les attributs ont un **type** qui ne change pas une fois l'attribut déclaré.

- ✍ Il ne vous reste plus qu'à dire au programme que lorsqu'un chien aboie, son nom s'affiche avant le message.

Pour cela, modifiez la méthode `bark()` :

```
void bark() {  
    System.out.println(this.name + " a dit : Wouf !");  
}
```

Observez ici :

- ▷ Le mot-clé `this` fait référence à l'objet sur lequel on appelle la méthode
- ▷ La notation `this.name` indique qu'on accède à l'attribut `name` de cet objet
- ▷ La concaténation de chaînes de caractères, en Java, se note avec l'opérateur `+`

Comment faire à présent pour donner un nom à chaque chien ?

Constructeur

On va donner à chaque chien son nom au moment où on *crée* l'objet qui lui correspond.

Jusqu'ici, on crée un objet de la classe `Dog` via :

```
Dog médor = new Dog();
```

Notez les parenthèses, qui indiquent des *arguments de méthode* !

Constructeur Lorsqu'on crée un objet, une méthode spéciale appelée *constructeur* est appelée. Le constructeur réserve la place en mémoire pour contenir l'objet, et initialise ses attributs.

En Java, un constructeur est une méthode qui a pour nom *le même nom que la classe*.

Si aucun constructeur n'est défini dans la classe, Java fournit un *constructeur par défaut* (sans argument). Si la classe possède un (ou plusieurs) constructeurs, le constructeur par défaut n'est plus utilisable.

- ✍ Ajoutez un constructeur de chien.

Dans la classe `Dog`, ajoutez une nouvelle méthode : le constructeur. Celui-ci reçoit le nom à donner au chien en paramètre, et stocke ce nom dans l'attribut `name` de l'objet créé.

```
Dog(String name) {  
    this.name = name;  
}
```

Observez en particulier la déclaration des arguments dans l'en-tête du constructeur !

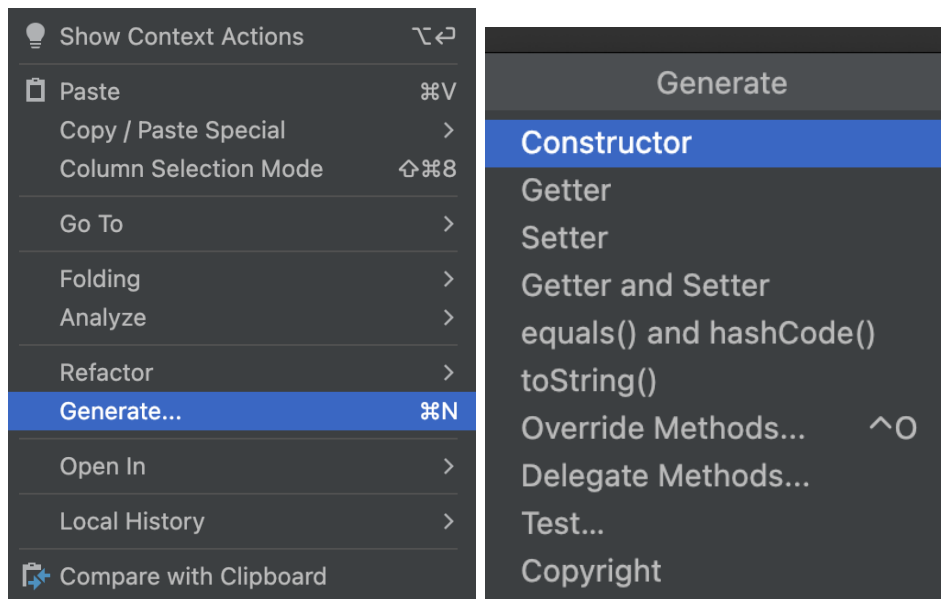
Arguments de méthode en Java Lorsqu'on définit une méthode en Java, les arguments se notent entre parenthèses, après le nom de la méthode. Chaque argument est précédé d'un *type*, de même que les variables et attributs. Une fois l'argument déclaré, il peut être utilisé au sein de la méthode comme s'il s'agissait d'une variable.

Notez en outre la notation `this.name = name` : le mot `name` est employé deux fois !

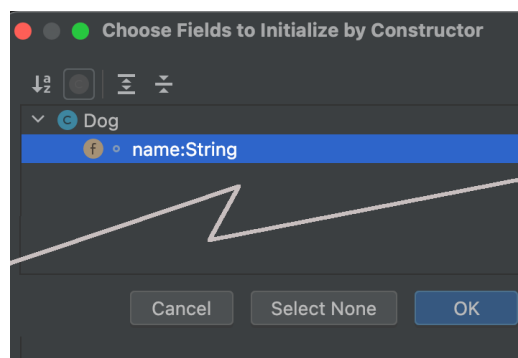
- ▷ `this.name` : comme il est précédé de `this.`, le compilateur Java devine qu'il s'agit d'une référence à un attribut
- ▷ `name` : puisqu'il est seul, l'argument de méthode (qui est définie de façon locale au constructeur) a priorité. Le compilateur décide donc qu'il s'agit d'une référence à l'argument du constructeur.

Ce type de constructeur est très commun ! Par conséquent, pour vous aider, IntelliJ offre un raccourci. Si vous faites un clic droit à l'intérieur du code de la classe, un menu s'ouvre avec l'option "Generate..." (Cette option se trouve aussi dans le menu "Code" au sommet de la fenêtre IntelliJ).

Parmi les options proposées se trouve "Constructor".



Ceci ouvre une boîte de dialogue où vous pouvez sélectionner *quels* attributs seront initialisés directement à partir d'une valeur reçue en argument. Ici, vous pouvez laisser `nom` sélectionnée et IntelliJ crée le constructeur pour vous.



Attention : Ceci n'est pas adapté à tout constructeur ! *Réfléchissez* à quels attributs vous voulez vraiment donner en paramètre du constructeur !

Remarque

Le constructeur généré aura une visibilité `public`, c'est très bien aussi (nous y reviendrons).

✎ Des messages d'erreurs apparaissent !

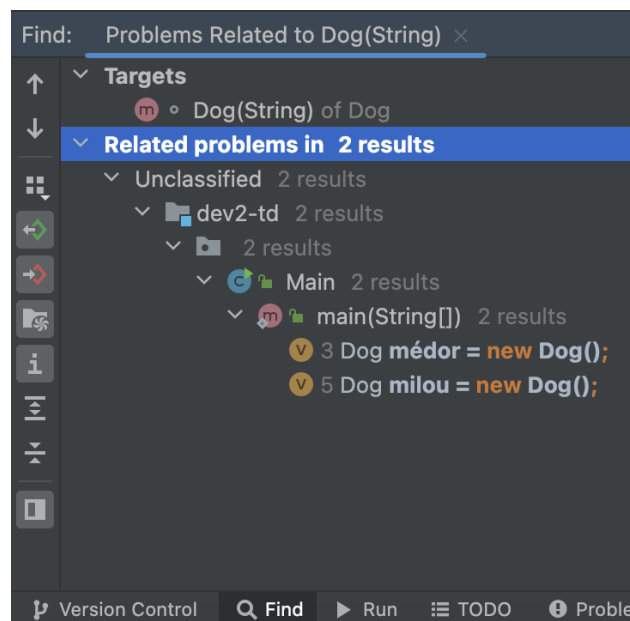
Que vous ayez écrit le constructeur à la main ou laissé IntelliJ le générer, un message d'erreur devrait apparaître dans votre interface :

```
2 usages  2 related problems
Dog(String nom) {
    this.nom = nom;
}
```

"Related problems" est un message d'erreur créé par IntelliJ pour vous informer que quelque part dans votre projet, cette méthode est appelée d'une façon qui n'est pas compatible avec la définition qui est faite ici.

Dans ce cas-ci, le problème vient du fait que nous avons défini un constructeur dans la classe `Dog` alors qu'il n'y en avait pas auparavant. Java pouvait donc utiliser un constructeur par défaut, mais maintenant qu'un constructeur est défini, ce n'est plus possible.

Si vous cliquez sur le message d'erreur, une interface "Find" s'ouvre en bas de l'écran :



Cela permet de trouver rapidement les erreurs quand le projet gagne en complexité.

Dans ce cas-ci, rendez-vous dans la classe `Main` et corrigez les instantiations de vos chiens :

```
public static void main(String[] args) {
    Dog médor = new Dog("Médor");
    médor.bark();
    Dog milou = new Dog("Milou");
    milou.bark();
}
```

Lorsque vous exécutez ce code, vous devriez obtenir :

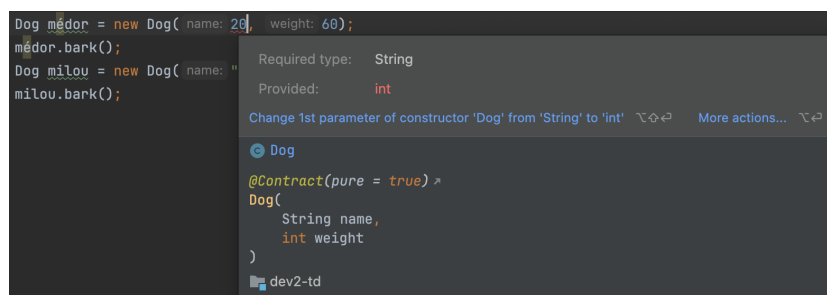
```
Médor a dit : Wouf !  
Milou a dit : Wouf !
```

```
Process finished with exit code 0
```

C'est ce que l'on souhaitait !

- ✍️ Faites une expérience : que se passe-t-il si on tente de renseigner une valeur du *mauvais* type ? Remplacez par exemple un des noms par un nombre entier. Immédiatement, IntelliJ souligne ce nombre en rouge : cela indique qu'une *erreur de compilation* a eu lieu. Puisque Java est compilé et fortement typé, les erreurs de type sont détectées sans même exécuter le programme !

Si vous placez votre souris sur la valeur incorrecte (*sans cliquer*), un pop-up s'ouvre :



IntelliJ vous donne des informations concernant l'erreur qui a eu lieu : dans ce cas-ci, qu'un élément de type **String** est attendu alors qu'un élément de type **int** a été fourni.

Vous avez maintenant les outils nécessaires pour doter chaque objet de caractéristiques qui lui sont propres via les *attributs* de cet objet, alors que le comportement est défini de façon commune pour tous les objets de la classe via ses *méthodes*.

3 Une fiche de contact

À vous de jouer ! Cet exercice vous fait créer une nouvelle classe en utilisant les concepts vus jusqu'ici.

Création de la classe

Créez une nouvelle classe **Contact**, qui possède quatre attributs :

- ▷ Le nom (*name*), prénom (*firstName*) et l'adresse postale (*address*), qui sont des chaînes de caractères.
- ▷ Le numéro de téléphone (*phone*), qui sera représenté par un nombre entier (**int**).

Écrivez ou générez un constructeur pour cette classe, qui reçoit une valeur pour chacun de ces attributs en argument.

Accesseurs

Tutoriel 2

Récupérer l'adresse

✍ Dans la classe `Contact`, écrivez une méthode `getAddress()`.

Cette méthode renvoie l'adresse contenue dans la fiche de contact concernée. Par conséquent,

- ▷ il s'agit d'une méthode dont le type de retour est une chaîne de caractères.
- ▷ elle renvoie la valeur d'un attribut.

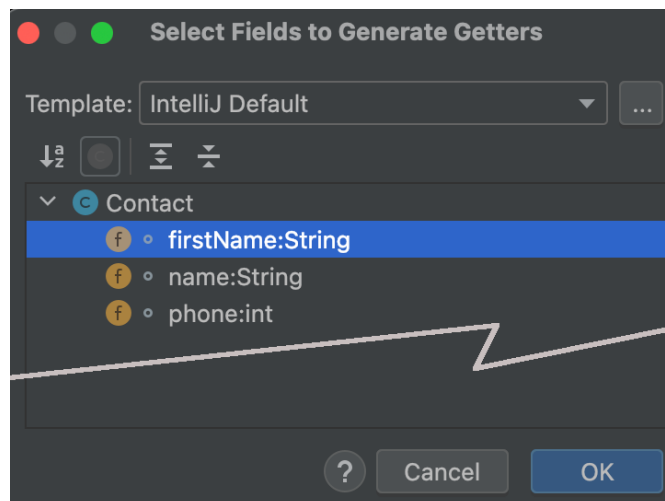
Votre code ressemblera probablement à

```
String getAddress() {  
    return this.address;  
}
```

On appelle ce type de méthode un *accesseur*.

Accesseur Un **accesseur** (*getter* en anglais) est une méthode qui accède à la valeur d'un attribut d'un objet donné, et renvoie simplement la valeur de cet attribut.

IntelliJ possède un raccourci pour générer des accesseurs ! Ouvrez de nouveau le menu "Generate"... et choisissez "Getter".



Une boîte de dialogue apparaît et vous montre la liste des attributs *qui ne disposent pas encore d'un accesseur*. Vous pouvez vous en servir afin de générer des accesseurs pour les autres attributs de `Contact`

Remarque

Si vous générez les accesseurs et en fonction de l'IDE, il générera `return this.address` ou `return address`. C'est équivalent dans ce contexte.

Autres méthodes en lecture

Bien souvent, on ne veut pas le nom ou prénom d'une personne mais bien son *nom complet* (prénom suivi du nom).

Écrivez une méthode `getFullName()` qui *renvoie* comme résultat le nom complet de la personne renseignée dans la fiche de contact.

Note : puisque cette méthode effectue une manipulation propre au contacts, IntelliJ ne vous proposera pas d'option pour la générer automatiquement. Ecrivez-la donc à la main !

Mutateurs

Les informations de contact d'une personne peuvent toujours changer ! Il est important que notre programme nous permettent de les mettre à jour sans devoir créer une nouvelle fiche de contact à chaque fois.

Tutoriel 3

Modifier le numéro de téléphone

✍ Ecrivez une méthode `setPhone()` qui reçoit un nouveau numéro de téléphone en argument, et met à jour le téléphone renseigné dans la fiche de contact. C'est donc une méthode qui :

- ▷ n'a pas de résultat (type de retour `void`)
- ▷ possède un argument
- ▷ assigne la valeur de cet argument à l'attribut `phone`

Votre code ressemblera sans doute à

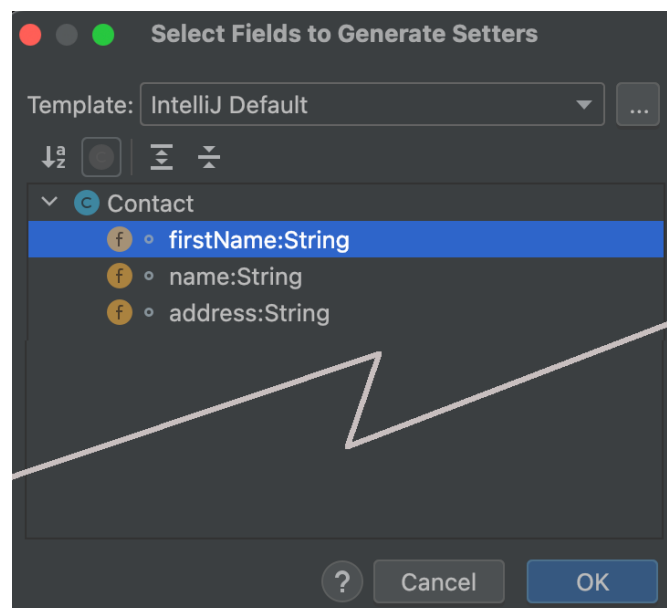
```
void setPhone(int phone) {  
    this.phone = phone;  
}
```

Notez que cette méthode ressemble à une des lignes du constructeur que vous avez sans doute écrit !

Ce type de méthode est ce qu'on appelle un *mutateur*.

Mutateur Un **mutateur** (*setter* en anglais) est une méthode qui reçoit une nouvelle valeur à assigner à un attribut de l'objet.

Tout comme pour les accesseurs, il est possible de laisser IntelliJ générer vos mutateurs : ouvrez le menu "Generate..." et sélectionnez "Setter".



Puis sélectionnez les attributs et générez votre code.

4 Retour à la classe Dog

Reprenons la classe `Dog` : nous allons ajouter deux fonctionnalités

Une question de taille

- ▷ Ajoutez un attribut `weight` dans la classe `Dog`. Cet attribut est un nombre entier représentant le poids en kilos, et il reçoit sa valeur dans le constructeur de la classe.

Ajoutez-aussi un poids à nos deux chiens : par exemple 60 pour `médor` et 7 pour `milou`.

- ▷ Modifiez la méthode `bark` afin que l'abolement effectué dépende du poids du chien. Par exemple, si le poids est supérieur à 30 kilos, le chien aboie avec `WOUUUF !`. S'il est inférieur à 10 kilos, il aboie avec `Yip yip!`. Sinon, l'abolement reste le `Wouf !` initial.

Gardez le fait que le nom du chien s'affiche dans tous les cas.

Condition en Java En Java, une condition utilise les mots-clés `if`, `else if` et `else`. Après un `if`, une expression booléenne qui encode la condition doit suivre, *entre parenthèses*. Le code à exécuter dans chaque cas est placé entre accolades afin de définir un bloc de code.

```
if (/* Première condition */) {  
    // Premier cas  
} else if (/* Deuxième condition */) {  
    // Deuxième cas  
} else {  
    // Cas sinon  
}
```

Testez votre code ! Exécutez votre programme et vérifiez qu'il affiche bien ceci :

```
Médor a dit : WOUUUF !  
Milou a dit : Yip yip !
```

```
Process finished with exit code 0
```

Un peu d'exercice

Vous allez ajouter une nouvelle fonctionnalité à vos chiens : la possibilité de courir, et de manger.

Un chien ne peut courir que s'il n'a pas faim ; une fois qu'il a couru, il a faim. Par contre, il peut manger quoi qu'il arrive, et une fois qu'il a mangé, il n'a plus faim.

- ▷ Ajoutez un attribut `hungry`, qui dit si le chien a faim ou non. En Java, le type booléen se déclare avec le mot-clé `boolean`.

Cet attribut ne reçoit pas de valeur en paramètre du constructeur ! Un chien n'a *pas* faim au moment où il est créé.

Donner une valeur par défaut à un attribut Lorsqu'un attribut possède une valeur par défaut (sa valeur initiale est la même pour tout objet de la classe), vous pouvez renseigner cette valeur au moment de déclarer l'attribut en effectuant immédiatement une assignation.

```
class Dog {  
    boolean hungry = false;  
}
```

- ▷ Générez un accesseur `isHungry` de cet attribut. Vous pouvez le générer automatiquement. Notez que, pour un attribut booléen, on appellera son accesseur *is...* au lieu de *get...*
- ▷ Ajoutez une méthode `run`. Si le chien a faim, cette méthode affiche un message **X a faim !** (ou X est le nom du chien). Si le chien n'a pas faim, cette méthode affiche un message **X court comme un fou !**, et le chien a maintenant faim.
- ▷ Ajoutez une méthode `eat`. Lorsque cette méthode est appelée, le chien n'a plus faim.

Testez votre code ! Ajoutez quelques manipulations dans votre méthode `main` afin de *tester* votre code. Par exemple, essayez d'afficher ceci :

```
Médor a dit : WOUUUF !  
Milou a dit : Yip yip !  
Médor court comme un fou !  
Milou court comme un fou !  
Milou a faim !  
Milou court comme un fou !
```

```
Process finished with exit code 0
```